U04 · Clases y objetos

Hasta ahora has usado clases ya hechas (`String`, `Scanner`, `Math`…). Es hora de **crear las tuyas propias** — el corazón de todo lo que viene después en este curso.

1. Los cuatro pilares de la POO

- **Abstracción**: simplificar la realidad, quedándote solo con lo esencial para tu problema. Al diseñar una clase `Persona` para una app de gestión académica, no necesitas su color de ojos — sí su nombre y sus notas.
- **Encapsulamiento**: agrupar datos (atributos) y comportamiento (métodos) en un mismo componente, y ocultar los detalles internos tras una interfaz pública. En Java no existen variables ni funciones sueltas fuera de una clase.
- **Herencia**: una clase puede especializar a otra, heredando sus atributos y métodos (la veremos a fondo en la unidad 5).
- **Polimorfismo**: un mismo método puede comportarse de forma distinta según el objeto o los parámetros con los que se use. Por ejemplo, `partida.empezar(4)` y `partida.empezar(rojo, azul)` son dos formas distintas de "empezar" una partida.

2. Definir una clase

```
[acceso] class NombreDeClase {
    [acceso] [static] tipo atributo1;
    [acceso] [static] tipo atributo2;

    [acceso] [static] tipoDevuelto metodo1(parámetros) {
        // cuerpo del método
    }
}
```

Ejemplo: una clase `Persona` con dos atributos y varios métodos.

```java
public class Persona {
    private String nombre;
    private int edad;

    public void setNombre(String n) { nombre = n; }
    public void setEdad(int e) { edad = e; }
    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }

    public boolean esMayorEdad() {
        return edad >= 18;
    }
}
```

Cada clase se guarda en su propio fichero `.java` (con el mismo nombre que la clase). Los atributos se llaman también **variables de instancia**: cada objeto de la clase tiene su propia copia, independiente de los demás.

3. Visibilidad de los miembros de una clase

Modificador	Accesible desde…
`public`	Cualquier clase
`protected`	La propia clase, sus subclases, y el mismo paquete
(nada, por defecto)	Solo el mismo paquete
`private`	Solo la propia clase

A los miembros `public` de una clase (los que se pueden usar desde fuera) se les llama su **interfaz**. La buena práctica de encapsulamiento es: **atributos `private`**, y una **interfaz pública** de métodos (`getX()`/`setX()`, y otros) para acceder a ellos de forma controlada — nunca accedas directamente a un atributo desde fuera de su clase.

```java
Persona p = new Persona();
p.setEdad(16);          // correcto: a través de la interfaz pública
// p.edad = 16;          // ¡mal! edad es privado, no compila desde fuera
```

4. Instanciar objetos

Una clase es solo una plantilla — para usarla, tienes que crear **objetos** (instancias) con `new`:

```
Persona p1;                 // 1. declarar la variable de referencia
p1 = new Persona();          // 2. crear el objeto en memoria y asignarlo
// o en una sola línea:
Persona p2 = new Persona();
```

> ⚠ **Una variable objeto es una referencia, no el objeto**
>
> ```
> Persona p1 = new Persona();
> p1.setNombre("Ada");
> Persona p2 = p1;    // ¡OJO! esto NO crea una copia
> p2.setNombre("Grace");
> System.out.println(p1.getNombre());  // imprime "Grace", no "Ada"
> ```
>
> `p2 = p1` copia la **referencia**, no el objeto — `p1` y `p2` acaban apuntando al mismo objeto en memoria. Es exactamente el mismo fenómeno que ya viste con arrays: cualquier cambio hecho a través de `p2` también lo notas a través de `p1`, porque en realidad solo hay un objeto.

Para acceder a los atributos y métodos de un objeto concreto, se usa el **operador punto** (`.`): primero identificas el objeto, luego el miembro.

```
Persona p = new Persona();
p.setNombre("Alan");
System.out.println(p.getNombre());
```

5. Constructores

Un **constructor** es un método especial que se ejecuta automáticamente al crear un objeto con `new`. Tiene el mismo nombre que la clase, y nunca declara tipo de retorno (ni siquiera `void`):

```java
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;   // this.nombre = el atributo; nombre = el parámetro
        this.edad = edad;
    }
}

Persona p = new Persona("Ada", 16);
```

`this` es una referencia al propio objeto sobre el que se está ejecutando el método — imprescindible aquí porque el parámetro y el atributo se llaman igual, y `this.nombre` es la única forma de distinguir "el atributo del objeto" de "el parámetro recibido".

Si no escribes **ningún** constructor, Java te da uno **por defecto** (sin parámetros, que deja los atributos a su valor por defecto: `0`, `false`, `null`…). En cuanto defines tú un constructor, ese constructor por defecto deja de existir — si además quieres poder crear objetos sin parámetros, tendrás que escribirlo tú explícitamente.

6. Atributos `static` y `final`

Combinando `static` y `final` en un atributo obtienes cuatro comportamientos distintos:

	Sin `final`	Con `final`
Sin `static`	Normal: cada objeto tiene su propio valor, que puede cambiar	Constante *de instancia*: cada objeto fija su valor en el constructor y ya no cambia
Con `static`	Compartido por todos los objetos de la clase, y puede cambiar	Constante *de clase*: un único valor, compartido y fijo — lo más habitual para constantes (`Math.PI`, por ejemplo, es `public static final`)

> ⚠ **Cuidado con abusar de static**
>
> Un atributo `static` es, en la práctica, una variable global disfrazada — rompe el encapsulamiento (cualquier objeto puede afectar a los demás a través de él) y es una fuente frecuente de bugs difíciles de rastrear. Resérvalo para casos donde de verdad tenga sentido un valor único compartido por todos los objetos de la clase.

7. Más sobre métodos: sobrecarga y sobrescritura

Ya viste la **sobrecarga** (*overloading*) en la unidad 2: varios métodos con el mismo nombre en la misma clase, distinguibles por su lista de parámetros.

```
public Persona(String nombre) { this(nombre, 0); }              // sobrecarga:
llama a otro constructor con "this(...)"
public Persona(String nombre, int edad) { this.nombre = nombre; this.edad = edad; }
```

La **sobrescritura** (*overriding*) es distinta: ocurre cuando una **subclase** redefine un método que ya tenía su **superclase**, con la misma firma exacta. La veremos con más profundidad al llegar a la herencia (unidad 5), pero ya vas a usarla en este mismo tema, al sobrescribir los métodos que toda clase hereda de `Object`.

8. La clase `Object`: la raíz de todo

En Java, **toda clase es subclase de otra**, con una única excepción: `Object`. Si no indicas explícitamente que tu clase hereda de otra, automáticamente es hija de `Object` — es el antepasado común de todos los objetos de un programa Java.

`Object` define varios métodos que **todas** las clases heredan, entre ellos:

Método	Qué hace	¿Deberías sobrescribirlo?
`String toString()`	Representación en texto del objeto	Sí, casi siempre

Método	Qué hace	¿Deberías sobrescribirlo?
`boolean equals(Object o)`	Compara si dos objetos son “iguales” en contenido	Sí, casi siempre
`int hashCode()`	Un resumen numérico del objeto (ligado a `equals`)	Sí, si sobrescribes `equals`
`Object clone()`	Copia superficial del objeto	Solo si lo necesitas

Sobrescribir `toString()`

Por defecto, `toString()` devuelve algo tan poco útil como `Persona@1b6d3586` (el nombre de la clase + un código hash). Sobrescríbelo para que tus objetos se muestren de forma legible:

```
@Override
public String toString() {
    return "Persona{nombre=" + nombre + ", edad=" + edad + "}";
}
```

A partir de ahí, `System.out.println(p)` o `"Persona: " + p` usan automáticamente tu versión.

Sobrescribir `equals()`

`==` sobre objetos compara si son **el mismo objeto en memoria** (igual que ya viste con `String` y con arrays), no si tienen el mismo contenido. Para comparar contenido, sobrescribe `equals()`:

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Persona)) return false;
    Persona otra = (Persona) obj;
    return this.edad == otra.edad && this.nombre.equals(otra.nombre);
}
```

Un `equals()` correcto debe cumplir tres propiedades: **reflexiva** (`x.equals(x)` siempre `true`), **simétrica** (`x.equals(y)` igual que `y.equals(x)`) y **transitiva** (si `x.equals(y)` y `y.equals(z)`,

entonces `x.equals(z)`).

⭐ **Be the Code**: si sobrescribes `equals()`, sobrescribe también `hashCode()` — Java exige que dos objetos "iguales" según `equals()` tengan siempre el mismo `hashCode()`. Si no lo haces, tus objetos se comportarán de forma incorrecta dentro de colecciones como `HashMap` o `HashSet` (unidad 6).

9. Clases inmutables

Una clase es **inmutable** cuando, una vez creado un objeto, su estado no puede cambiar nunca. Esto evita toda una categoría de errores (sobre todo en programas con varios hilos ejecutándose a la vez) porque nadie puede modificar el objeto "por sorpresa" — cualquier "cambio" implica crear un objeto nuevo (es justo el caso de `String`, que ya conoces).

Para que una clase sea realmente inmutable:

- Todos sus atributos son `private final`.
- No tiene *setters* ni ningún método que cambie su estado.
- Si recibe o devuelve objetos mutables (como un array), hace una **copia defensiva** — nunca comparte la referencia original.

```
public final class AlumnoInmutable {
    private final int id;
    private final String nombre;
    private final int[] notas;

    public AlumnoInmutable(int id, String nombre, int[] notas) {
        this.id = id;
        this.nombre = nombre;
        this.notas = notas.clone();       // copia defensiva al ENTRAR
    }

    public int[] getNotas() {
        return notas.clone();              // copia defensiva al SALIR
    }
    // getId(), getNombre()...
}
```

Sin las dos copias defensivas (`.clone()`), cualquiera que tuviera acceso al array original (o al que devuelve el getter) podría modificar las notas del alumno desde fuera — rompiendo la promesa de inmutabilidad aunque los atributos sean `final`.

10. El patrón Singleton

Una clase **Singleton** garantiza que, en toda la aplicación, **solo puede existir un objeto** de esa clase. Se consigue haciendo el constructor `private` (para que nadie pueda usar `new` desde fuera) y ofreciendo un método estático que crea el objeto la primera vez que se pide, y devuelve siempre esa misma instancia después:

```
public class Configuracion {
    private static Configuracion instancia;

    private Configuracion() { /* ... */ }   // constructor privado: nadie más lo
llama

    public static Configuracion getInstancia() {
        if (instancia == null) {
            instancia = new Configuracion();
        }
        return instancia;
    }
}

Configuracion c1 = Configuracion.getInstancia();
Configuracion c2 = Configuracion.getInstancia();
// c1 y c2 son, literalmente, el mismo objeto
```

Es útil para representar algo de lo que, por su propia naturaleza, no debería haber más de una instancia — una configuración global, una conexión compartida, un registro de logs.

11. Paquetes

Un **paquete** (*package*) es una carpeta que agrupa clases relacionadas — la forma que tiene Java de organizar un proyecto grande y evitar colisiones de nombres entre clases de distintas librerías.

```
package com.miinstituto.gestion;   // primera línea del fichero: a qué paquete pertenece

import java.util.Scanner;          // importa una clase de OTRO paquete para poder usarla
```

Dentro del mismo paquete, las clases y sus miembros con visibilidad “por defecto” (sin modificador) ya son accesibles entre sí — por eso hasta ahora no has necesitado ningún `import` para tus propias clases del mismo proyecto.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA1. Reconoce la estructura de un programa informático…	a) estructura de un programa · d) tipos de variables · f) conversión de tipos
RA2. Utiliza estructuras de control y datos…	a) fundamentos POO · b) programas simples · c) instanciación de objetos · d) métodos y propiedades · e) métodos estáticos · f) parámetros en llamadas · g) librerías de objetos · h) constructores · i) IDE · j) *(uso general del entorno)*
RA4. Desarrolla programas utilizando objetos y clases…	a) sintaxis y estructura de una clase · b) definir clases · c) propiedades y métodos · d) constructores · f) visibilidad

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras